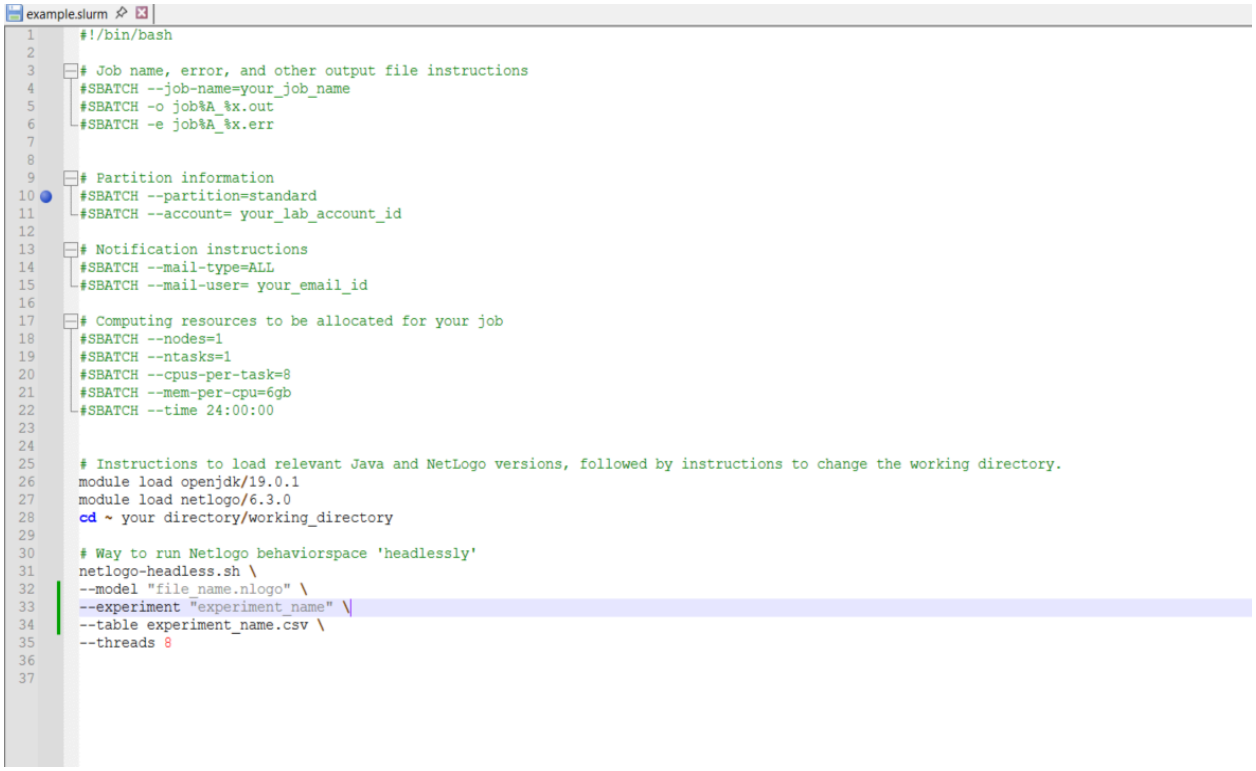
SLURM file code explanation

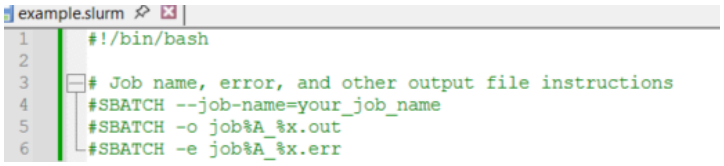
This document gives a detailed overview of what each line in the SLURM script (File name: example.slurm) means and how you can customize this to run specific experiments using the behaviorspace tool in NetLogo.

SLURM script screenshot (example.slurm)



```
#!/bin/bash

# Job name, error, and other output file instructions
#SBATCH --job-name=your_job_name
#SBATCH -o job%A_%x.out
#SBATCH -e job%A_%x.err


# Partition information
#SBATCH --partition=standard
#SBATCH --account= your_lab_account_id

# Notification instructions
#SBATCH --mail-type=ALL
#SBATCH --mail-user= your_email_id

# Computing resources to be allocated for your job
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem-per-cpu=6gb
#SBATCH --time 24:00:00


# Instructions to load relevant Java and NetLogo versions, followed by instructions to change the working directory.
module load openjdk/19.0.1
module load netlogo/6.3.0
cd ~ your directory/working_directory

# Way to run Netlogo behaviorspace 'headlessly'
netlogo-headless.sh \
--model "file_name.nlogo" \
--experiment "experiment_name" \
--table experiment_name.csv \
--threads 8
```

Line-wise breakdown of the SLURM script



```
#!/bin/bash

# Job name, error, and other output file instructions
#SBATCH --job-name=your_job_name
#SBATCH -o job%A_%x.out
#SBATCH -e job%A_%x.err
```

Explanation: The first line is extremely essential, and it indicates that the bash language is being used. The second line above provides the job name. This is how it will typically appear on the HPC queue. The third and fourth lines are for debugging- the second line instructs HPC to create this file when the job starts, and the third line is to create an error file. In my opinion, the third line is more important than the second one. The error file can be used to track if there is something wrong with your job or an error in the SLURM file itself.

IMP note: '#' is used for writing comments/notes in the script, which will not be run. However, '#SBATCH' is recognized as a script and not a comment.

```
# Partition information
#SBATCH --partition=standard
#SBATCH --account= your_lab_account_id
```

Explanation: Each lab at my university is allotted a certain number of CPU hours every month, and when you tell HPC to use the 'standard' partition, it will use those CPU hours. If you use 'partition= windfall', it will use windfall hours. Windfall hours are unlimited, but the main issue is that those jobs are not prioritized, and if there is another job that requires standard hours, your job will be paused/terminated. However, the way the partitions are defined may differ across institutions. So, do check with your institute/university as to how this is defined and edit accordingly.

```
# Notification instructions
#SBATCH --mail-type=ALL
#SBATCH --mail-user= your_email_id
```

Explanation: These two lines are for indicating whether you want to be notified via email of the progress of your HPC job. If you are giving too many jobs at once, I would recommend deleting these lines to avoid inundating your inbox. Again, this may differ across institutions, so do check and edit accordingly.

```
# Computing resources to be allocated for your job
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem-per-cpu=6gb
#SBATCH --time 24:00:00
```

Explanation: These lines specifically indicate the computing resources for your job.

- The first line here indicates how many nodes or computers you want for your job to run. Unless you know that there is an explicit parallel processing or ability to run your job in parallel on multiple computers, you should just use one node. From what I can tell, NetLogo does not have this ability.
- Second and third lines are about how many tasks/runs you want to run at a given time, and how many cpus do you want per run to be used (**Total CPUs that will be used** = ntasks x cpus-per-task). This part does need a little fine-tuning, and it will depend on the computing power and limits of the clusters at your institute/university. Within NetLogo Behaviorspace, one can run multiple parallel runs/tasks (**IMP:** that they are not really parallel in a literal sense of parallel computing, but they will run in parallel on different cores of the same node/computer), which can be given as 'ntasks').
- The fourth line is how much memory/RAM you want for each of the CPUs. At my university's cluster, allocation of 6gb is optimal (anything beyond was not improving the performance). Again, this needs a bit of fine-tuning.
- The fifth line is the total clock time that you want the job to run. Now the CPU hours are calculated by multiplying the clock time by the total number of CPUs. So here, total CPU

hours are = 8 x 24, which is 192 CPU hours (which will be deducted from the allocated CPU hours from your lab's account).

```
# Instructions to load relevant Java and NetLogo versions, followed by instructions to change the working directory.
module load openjdk/19.0.1
module load netlogo/6.3.0
cd ~ your directory/working_directory
```

module load openjdk/19.0.1
module load netlogo/6.3.0
cd ~ your directory/working_directory

Explanation: The first two lines indicate that you want HPC to use module 'openjdk' (Java) and the specific 19.0.1 Java version, and use the specific NetLogo version (I used 6.3.0 for my work). The specific way in which the Java and NetLogo versions are referred to in the SLURM script may change depending on your institute/university's HPC. The third line indicates that you are changing the working directory to another file (here, you are changing it to a folder called 'working_directory'). **IMP:** This folder contains the NetLogo file and the SLURM file. The output file (and the error files) will also be saved within this folder.

```
# Way to run Netlogo behaviorspace 'headlessly'
netlogo-headless.sh \
--model "file_name.nlogo" \
--experiment "experiment_name" \
--table experiment_name.csv \
--threads 8
```

Explanation: These lines are for running NetLogo Behaviorspace experiments on HPC. This part has been modified using documentation on NetLogo's official website. NetLogo uses the netlogo-headless.sh file to run NetLogo Behaviorspace on HPC clusters/command line without the GUI.

- The first line here tells that netlogo-headless is being run.
- The second line tells what the NetLogo file (i.e., your specific model file) needs to be run (which is within your working directory).
- The third line tells which specific behaviorspace experiment needs to be run.
- The fourth line is the output file, its name, and format. Table format is the best format in terms of the file being almost data-analysis ready (And requires minimal to no cleaning).
- **IMPORTANT:** The fifth line indicates the number of threads being used. The CPU allocation above just allocates the job on HPC; it does not tell how the job would be run on HPC. The thread number tells HPC to run those many jobs at the single time (one thread for each CPU), so here 8 jobs will be run at once (1 per CPU). As per my experience, if this line is not given, there may be just one job run at a time, and the CPU hours are wasted! Also, from my experience, the number of threads should match the total number of CPUs for the job to run smoothly. I have tried using more threads or fewer threads than the number of CPUs, and the jobs seem to run very slowly (and the CPU efficiency is very low in that case). This also may require a bit of fine-tuning.

A few tips on optimizing the computing resources to be allocated:

- It is best to first run a BehaviorSpace experiment with very few simulation runs,~50-100, to get an idea of how much time each run needs.
- From my experience, it is better to run a large number of jobs with a smaller number of runs rather than give one single job with a large number of runs. One reason is the lack of sufficient computing resources available. Since you are not the only one running jobs on HPC (unless you have your private cluster, then good for you!), it may take a long time for your job to start. Secondly, you can get the same job done faster. Let me give an example. Say you are running a NetLogo experiment with 2000 runs. Rather than giving one single job of 2000 runs, give 10 identical jobs with 200 runs each. This way each single job will be done much faster. You can use 'array jobs' to make your job easier.
- If you run out of time before all simulation runs are executed, the job will be aborted. But note that you can still download the resulting .csv file, which will have all the runs up to the abort in it. So, it is always best to give 3-5 hrs of extra cushion time! For unknown reasons, the same job takes slightly different times when run again (with the same specifications). At least at my university, the unspent time is returned to the account (Do check with your institute/university's HPC team if this is true for you as well).
- If too many CPUs are assigned to the job, there is a problem with the Java heap. NetLogo headless uses Java to run, and there needs to be optimum CPUs and memory assigned in order for the java to clear the 'garbage'. For example, I tried using 80 CPUs with 5 GB of memory per CPU for 540 runs for one of the experiments, and there was a Java heap error (GC limit reached). So it is optimal to use fewer CPUs (thus more clock time).